

Final Project

Data-Driven Optimal Power Flow for Distribution Grids

ENEL 619.94

Machine Learning for Electrical Energy Systems

Schulich School of Engineering
University of Calgary

Important Deadlines

Project Mid-Point Check:

March 9, 2026, 11:59 PM MT

Final Report + Git Repository:

April 19, 2026, 11:59 PM MT

Oral Defense:

Week of April 20, 2026

Winter 2026

© 2026 Mostafa Farrokhhabadi

Contents

1 Project Overview	3
1.1 The Challenge	3
1.2 Key Skills Developed	3
2 Learning Outcomes	4
3 Background and Context	4
3.1 The IEEE 33-Bus System	4
3.2 The Optimal Power Flow Problem	5
4 Project Options	5
4.1 Option 1: Time-Series Forecasting + Regression-Based OPF	5
4.1.1 Key Considerations	5
4.2 Option 2: Time-Series Forecasting + RL-Based OPF	6
4.2.1 Key Considerations	6
4.3 Option 3: End-to-End Supervised Learning OPF	6
4.3.1 Key Considerations	6
4.4 Option 4: End-to-End Reinforcement Learning	7
4.4.1 Key Considerations	7
4.5 Choosing Your Approach	7
5 The Benchmark: Naive Forecasting + Batch OPF	8
5.1 Benchmark Description	8
5.2 Why This Benchmark?	8
5.3 Evaluation Metrics	8
6 Data and Access and Getting Started	9
6.1 Accessing the Data	9
6.2 Getting Started Guide	9
7 Mid-Point Check (Formative, Required to Pass)	10
8 Final Report (20/55 points)	10
8.1 Required Sections	10
8.2 Formatting Requirements	11
9 Code Repository and Reproducibility (15/55 points)	12
9.1 Required Repository Structure	12
9.2 README Requirements	13
9.3 Reproducibility Checklist	13
10 Oral Defense (20/55 points)	13
11 Assessment Rubric	14
11.1 Final Report (20/55 points)	14
11.2 Code Repository and Reproducibility (15/55 points)	15
11.3 Oral Defense (20/55 points)	16

12 Timeline and Milestones	16
12.1 Recommended Schedule	16
13 Academic Integrity and GenAI Use Policy	17
13.1 Generative AI Use Policy	17
13.2 Required Disclosure	17
13.3 Verification	17
14 Frequently Asked Questions	18
15 Data Confidentiality and Research Collaboration	19

1 Project Overview

This final project challenges you to design and implement a data-driven optimal power flow (OPF) engine for a distribution grid with solar PV and energy storage system (ESS). You will work with the IEEE 33-bus distribution system, the same test system used in Assignment 1, but now focusing on developing machine learning solutions for real-time operational decision-making.

1.1 The Challenge

The traditional model-based OPF solutions solve large non-linear, non-convex (or linearized/convexified approximate) optimization problems that may be computationally expensive for real-time deployment. Thus, Your task is to develop a data-driven approach that can:

- Learn from historical operational data
- Make real-time or near-real-time dispatch decisions
- Outperform a naive baseline benchmark
- Generalize to unseen operating conditions

Data-driven OPF represents a paradigm shift in power systems operations:

- **Computational efficiency:** Making decisions in milliseconds vs. minutes/hours
- **End-to-end learning:** Learning optimal policies without explicit forecasting

1.2 Key Skills Developed

- Translating power systems optimization problems into machine learning tasks with appropriate problem framing, loss functions, and evaluation metrics
- Designing and implementing either:
 - Time-series forecasting models
 - Supervised learning for sequential decision-making
 - Reinforcement learning algorithms
 - End-to-end learning architectures
- Ensuring learned policies satisfy physical constraints through projection methods, penalty approaches, or safe learning techniques
- Conducting systematic experiments with proper train/validation/test splits, ablation studies, and statistical analysis
- Research-Grade Development Practices:
 - Professional Git workflows with meaningful version control
 - Experiment tracking and hyperparameter management
 - Reproducible research with documentation and configuration management
 - Code testing and validation

- Understanding OPF formulation, energy storage dynamics, and distribution grid constraints
- Comparing ML approaches against model-based baselines with fair evaluation protocols
- Writing IEEE-format technical papers, creating professional presentations, and defending technical decisions in oral examinations
- Analyzing limitations and failures, and proposing improvements based on empirical results
- Team work on technical projects with distributed responsibilities and integrated deliverables
- Technical communication through code, reports, and presentations

Working Format

This project must be completed in **fixed groups of two students**.

2 Learning Outcomes

By completing this project, you will be able to:

1. **Formulate** power systems optimization problems as machine learning tasks
2. **Design** and implement ML pipelines for time-series forecasting in power systems
3. **Implement** supervised learning or reinforcement learning approaches for OPF
4. **Evaluate** ML model performance using appropriate power systems metrics
5. **Compare** data-driven approaches against model-based baselines
6. **Communicate** technical ML research through code, documentation, and oral defense
7. **Collaborate** on a research-grade ML project using professional workflows

3 Background and Context

3.1 The IEEE 33-Bus System

You will use the same modified IEEE 33-bus distribution system¹ from Assignment 1:

- 33 buses in radial configuration
- Distributed PV generation at all buses scaled to 20% penetration
- Single ESS - location and capacity defined in dataset
- Normalized load profiles for 4 years that must be scaled to the system nominal loading
- Real PV generation profiles over 1 year. You can assume the same PV profile over multiple years. Throughout, assume access to perfect PV forecast

¹<https://ieeexplore.ieee.org/document/9258930>

3.2 The Optimal Power Flow Problem

The OPF problem aims to minimize system power losses over a 24-hour horizon while:

- Maintaining bus voltages within limits (0.9 - 1.1 p.u.)
- Respecting ESS power and energy constraints
- Satisfying power balance at all buses

The mathematical formulation is based on the Branch Flow model². A reference implementation is provided in the project repository. You may code a non-linear AC OPF as the expert, but that is not necessary for completion of this project.

4 Project Options

You must choose **ONE** of the following four approaches for your project. Each represents a different philosophy for integrating machine learning with power systems optimization.

4.1 Option 1: Time-Series Forecasting + Regression-Based OPF

This two-stage approach separates forecasting from optimization:

1. Stage 1 - Load Forecasting:

- Train a time-series forecasting model to predict future load
- Forecasting horizon: your choice (e.g., 1-hour, 6-hour, 24-hour ahead)
- Input features: historical load and time features; temperature (optional)

2. Stage 2 - Regression-Based OPF:

- Train a regression model to optimize ESS dispatch decisions over a horizon of your choice.
- Inputs: forecasted load, PV generation, current ESS state-of-charge, time features
- Outputs: ESS active and reactive power setpoints
- Training data: optimal ESS schedules from solving OPF on historical data

4.1.1 Key Considerations

- How to handle forecasting errors propagating to dispatch decisions?
- How to ensure physical constraints (power limits, SoC bounds) are satisfied?
- What forecasting horizon provides best trade-off between accuracy and lookahead?

²<https://ieeexplore.ieee.org/abstract/document/6507355>

4.2 Option 2: Time-Series Forecasting + RL-Based OPF

Combines forecasting with reinforcement learning for sequential decision-making:

1. Stage 1 - Load Forecasting:

- Same as Option 1

2. Stage 2 - RL-Based OPF:

- Formulate ESS dispatch as a Markov Decision Process (MDP)
- State: current load, PV generation, ESS SoC, forecasted conditions
- Action: ESS active/reactive power setpoints
- Reward: design as appropriate.

4.2.1 Key Considerations

- How to design the reward function to balance losses vs. constraint satisfaction?
- How to handle continuous action spaces (ESS power)?
- How to ensure exploration doesn't violate critical constraints?

4.3 Option 3: End-to-End Supervised Learning OPF

Direct mapping from current observations to optimal dispatch without explicit forecasting:

- **Input:** Current and recent historical observations
 - Current load at all buses
 - Current PV generation
 - ESS state-of-charge
 - Time features (hour, day of week, season)
 - Recent load/PV trajectories (sliding window); temperature (optional)
- **Output:** ESS dispatch decision for current time step
- **Training:** Learn from optimal solutions of historical OPF problems

4.3.1 Key Considerations

- Can the model learn to implicitly forecast from historical patterns?
- How much historical context (window length) is needed?
- How to enforce physical constraints in the learned policy?
- How does performance compare to explicit forecasting approaches?

4.4 Option 4: End-to-End Reinforcement Learning

Pure RL approach without separate forecasting module:

- **State:** Current observations + recent history
 - Same as Option 3 inputs
- **Action:** ESS power setpoints
- **Reward:** Design as appropriate

4.4.1 Key Considerations

- How to design state representation to capture temporal dependencies?
- How to balance exploration vs. constraint satisfaction?
- How to handle non-stationarity in load/PV patterns?
- Sample efficiency: how much training data is needed?

4.5 Choosing Your Approach

Option	Advantages	Challenges
Forecasting + Regression	Higher interpretability	Forecasting; constraint handling
Forecasting + RL	Exploration	Forecasting; complex training
End-to-End Supervised	No forecasting	Sample inefficiency; complex training
End-to-End RL	No forecasting; exploration	Sample inefficiency; complex training

Table 1: Comparison of project options

Recommendation

Choose based on your interests and background:

- Interested in time-series forecasting? → Options 1 or 2
- Interested in deeper MLP? → Options 3 or 4
- Interested in RL? → Options 2 or 4
- Want most interpretable approach? → Option 1
- Want most research novelty? → Options 4

5 The Benchmark: Naive Forecasting + Batch OPF

You must outperform the following baseline approach:

5.1 Benchmark Description

1. Naive 24-Hour Ahead Load Forecast:

- Prediction: Tomorrow's load = Today's load
- Formally: $\hat{L}(t + 24h) = L(t)$ for all hours
- No learning, no sophisticated forecasting

2. 24-Hour Batch OPF:

- Solve full Branch Flow OPF once per day
- Uses naive forecast for next 24 hours
- Optimizes ESS dispatch for entire 24-hour period
- Updates once every 24 hours

5.2 Why This Benchmark?

This represents a simple but reasonable baseline that:

- Uses model-based optimization (OPF)
- Requires minimal computational overhead (1 solve per day)
- Makes a simple assumption about future load
- Provides a clear target to beat

Benchmark results will be provided.

5.3 Evaluation Metrics

Your approach will be compared against the benchmark using:

1. Primary Metric - Total System Losses:

$$\text{Total Losses} = \sum_{t=1}^T \sum_{(i,j) \in \mathcal{L}} r_{ij} i_{ij}^2(t) \quad (1)$$

- Performance on held-out test data (unseen days)
- Lower is better; your goal is to achieve lower total losses than the benchmark

2. Constraint Violation Rate:

$$\text{Violation Rate} = \frac{\text{Number of constraint violations}}{\text{Total number of time steps}} \quad (2)$$

Constraints include voltage limits and ESS power/energy limits. Must be $< 5\%$.

3. Computational Time:

- Average inference/decision time per time step
- Must be suitable for real-time operation (e.g., < 1 second)

6 Data and Access and Getting Started

6.1 Accessing the Data

1. Download the dataset and code from <https://github.com/ENEL-619-L03-W-2026/Final-Project>
2. Review the provided data organization recording under “Handout & Instructions”

6.2 Getting Started Guide

Step 1: Set up your development environment

Option A - Use Provided Environment:

```

1 # Create environment from provided file
2 conda env create -f environment.yml
3
4 # Activate the environment
5 conda activate enel619
6
7 # Verify IPOPT installation
8 from pyomo.environ import SolverFactory
9 SolverFactory('ipopt').available()

```

Listing 1: Create Environment from YAML File

Option B - Manual Installation:

```

1 # Create new Conda environment
2 conda create -n myenv python=3.11
3
4 # Activate your Conda environment
5 conda activate myenv
6
7 # Install required packages
8 conda install pyomo
9 conda install -c conda-forge ipopt
10
11 # Verify IPOPT is installed
12 from pyomo.environ import SolverFactory
13 SolverFactory('ipopt').available()

```

Listing 2: Install Required Packages Manually

Step 2: Explore the provided sample code

- Understand how data is loaded and processed
- Run the OPF code

7 Mid-Point Check (Formative, Required to Pass)

Due: March 9, 2026, 11:59 PM MT

Submit via D2L:

1. **Project Proposal** (1 pages PDF):

- Chosen approach (Option 1, 2, 3, or 4)
- Literature review: 3-5 relevant papers on your chosen approach
- Proposed methodology and architecture
- Preliminary implementation plan
- Division of responsibilities between team members

2. **Git Repository Link:**

- Repository initialized with proper structure

3. **Preliminary Results** (optional):

- Baseline benchmark results on validation set
- Initial model training results
- Early experiments

Mid-Point Check is Required

This checkpoint is formative and not numerically graded. However, **failure to complete it satisfactorily may result in a failing grade for the entire project.**

8 Final Report (20/55 points)

Due: April 19, 2026, 11:59 PM MT

Submit as PDF via D2L using the IEEE PES Transactions template.

8.1 Required Sections

1. **Abstract** (150-200 words)

2. **Introduction:**

- Problem motivation
- Research question
- Contribution summary

3. **Background and Related Work:**

- Review of data-driven OPF approaches
- Positioning of your work
- Minimum 10 relevant references

4. Methodology:

- Detailed description of your approach
- Model architecture
- Training procedure
- Hyperparameter selection
- Constraint handling strategy

5. Experimental Setup:

- Dataset description
- Benchmark description
- Evaluation metrics
- Implementation details

6. Results:

- Dataset description
- Generalization performance
- Constraint satisfaction analysis
- Comparison with benchmark
- Computational efficiency

7. Discussion:

- Interpretation of results
- Strengths and limitations
- Lessons learned
- Comparison with other approaches (if applicable)

8. Conclusions and Future Work**9. References** (IEEE format)**10. Appendices** (as needed):

- Additional results
- Hyperparameter tables
- Extended ablation studies

8.2 Formatting Requirements

- Template: IEEE PES Transactions (<https://ieee-pes.org/publications/authors-kit/templates-and-sample-of-pes-technical-papers/>)
- Length: 6 pages (excluding references and appendices)
- All figures must be high-resolution with clear captions
- All tables must be properly formatted
- All experimental claims must be supported by data

9 Code Repository and Reproducibility (15/55 points)

Due: April 17, 2026, 11:59 PM MT

Submit Git repository link via D2L.

9.1 Required Repository Structure

```

1 final-project-groupXX/
2     README.md                    # Project documentation
3     requirements.txt             # Python dependencies
4     environment.yml             # Conda environment
5     data/
6         README.md                # Data documentation
7         raw/                     # Original data
8         processed/               # Preprocessed data
9     src/
10         __init__.py
11         data/
12             loader.py            # Data loading utilities
13             preprocessing.py      # Data preprocessing
14         models/
15             forecasting.py        # Forecasting models
16             opf_model.py          # Your ML-based OPF model
17             baseline.py           # Benchmark
18 implementation
19     training/
20         train.py                 # Training script
21         utils.py                 # Training utilities
22     evaluation/
23         metrics.py               # Evaluation metrics
24         evaluate.py              # Evaluation script
25         power_flow.py            # Power flow solver
26     configs/
27         model_config.yaml        # Model hyperparameters
28         experiment_config.yaml   # Experiment settings
29     experiments/
30         train_model.py           # Main training script
31         evaluate_model.py        # Main evaluation script
32         run_benchmark.py         # Run benchmark
33     notebooks/
34         01_data_exploration.ipynb
35         02_baseline_analysis.ipynb
36         03_model_development.ipynb
37         04_results_visualization.ipynb
38     results/
39         figures/                 # Generated figures
40         metrics/                 # Saved metrics
41         models/                  # Trained model checkpoints
42     tests/
43         test_data.py             # Data processing tests
44         test_models.py           # Model tests

```

Listing 3: Required Repository Structure

9.2 README Requirements

Your README must include:

1. **Project Overview:**
 - Brief description of approach
2. **Installation Instructions:**
 - Environment setup (conda/pip)
 - Dependency installation
 - Data download/setup
3. **Usage Guide:**
 - How to reproduce training
 - How to reproduce evaluation
4. **Project Structure:**
 - Explanation of directory organization
 - Description of key files

9.3 Reproducibility Checklist

- ☐ All random seeds are set and documented
- ☐ Data preprocessing is fully scripted
- ☐ Model training is fully scripted
- ☐ Hyperparameters are stored in config files
- ☐ Evaluation metrics match those in report
- ☐ All figures in report can be regenerated
- ☐ Instructions are clear enough for someone else to reproduce
- ☐ Repository includes pretrained models or training completes in reasonable time

10 Oral Defense (20/55 points)

Week of April 20, 2026

Teams will have a **20-minute group presentation** and a **40-minute individual defense**.

1. **Presentation** (20 minutes):
 - Problem and motivation (1 min)
 - Your approach and methodology (10 min)
 - Key results and analysis (7 min)

- Conclusions and lessons learned (2 min)
2. **Q&A and Deep Dive** (40 minutes):
- Technical questions about your approach
 - Clarification on implementation details
 - Discussion of results and limitations
 - Code walkthrough (be prepared to show and explain code)

The individual defense will focus on the followings:

- **Technical depth:** Do you deeply understand your approach? Can you explain general ML/optimization concepts?
- **Implementation knowledge:** Can you explain your code?
- **Critical thinking:** Can you discuss limitations of your project and alternatives?

Attendance Requirement

Both members must attend the group presentation session. Missing the session results in 0 for the presentation component.

11 Assessment Rubric

11.1 Final Report (20/55 points)

Criterion	Excellent (20)	Satisfactory (12-19)	Must Improve (5-12)
Problem Formulation	Clear ML formulation; well-motivated design choices; comprehensive literature review	Good formulation; reasonable motivation; adequate literature	Basic formulation; limited motivation; minimal literature
Methodology	Detailed, clear methodology; novel insights; well-justified; reproducible	Clear methodology; standard approach; mostly justified	Basic methodology; some gaps; limited justification
Experimental Design	Comprehensive experiments; proper baselines; thorough ablations; statistical rigor	Good experiments; appropriate baselines; some ablations	Basic experiments; simple comparisons; limited analysis
Results & Analysis	Beats benchmark; insightful analysis; multiple perspectives; honest limitations	Beats or matches benchmark; good analysis; identifies limitations	Below benchmark or marginal; surface analysis; ignores limitations
Writing	Excellent clarity; professional figures; proper citations; well-organized	Good clarity; adequate figures; mostly proper citations	Acceptable clarity; basic figures; citation issues

Table 2: Final report rubric

11.2 Code Repository and Reproducibility (15/55 points)

Criterion	Excellent (15)	Satisfactory (10-15)	Must Improve (5-10)
Code Quality	Modular, clean, well-documented; follows best practices; professional-grade	Good structure; adequately documented; mostly clean	Basic structure; minimal documentation; some issues
Reproducibility	Fully reproducible; clear instructions; all seeds set; config-driven	Mostly reproducible; good instructions; some manual steps	Partially reproducible; unclear instructions; manual intervention needed
Experiment Tracking	Comprehensive logging; version control; organized results	Good logging; basic version control	Minimal logging; poor organization
Git Repository	Regular meaningful commits; clear commit messages; both members' contributions visible; proper .gitignore	Good commit history; adequate messages; contributions evident	Few commits; poor messages; unclear contributions or single bulk commit

Table 3: Code repository rubric

Code Quality Definitions

1. Modular design:

- Separate data processing, model training, evaluation
- Reusable components
- Clear abstractions

2. Documentation:

- Docstrings for all functions/classes
- README with setup and usage instructions
- Comments for complex algorithms

3. Experiment tracking:

- Log hyperparameters
- Save training curves
- Record random seeds
- Use tools like TensorBoard, Weights & Biases, or MLflow

4. Version control:

- Regular commits with meaningful messages
- Clear Git history showing both team members' contributions

11.3 Oral Defense (20/55 points)

Criterion	Points	Description
Presentation Quality & Delivery (group)	8	Clarity of presentation; accuracy; quality of insights; Speaking skills; engagement; timing; equal participation
Technical Understanding (individual)	8	Deep understanding of approach; can explain technical details; justifies design decisions
Implementation Knowledge (individual)	4	Can navigate and explain code; understands implementation; can discuss alternatives

Table 4: Oral defense rubric

12 Timeline and Milestones

12.1 Recommended Schedule

Date	Milestone
Mar 2-6	Literature review; choose approach; initialize repository
Mar 6-13	Data preprocessing; mid-point check submission
Mar 13 - Mar 27	Core model development; initial training experiments
Mar 27 - Apr 6	Model refinement; hyperparameter tuning; ablation studies
Apr 6 - Apr 10	Final experiments on test set; results analysis
Apr 10 - Apr 19	Report writing; code cleanup; repository documentation
Week of Apr 20	Oral defenses

Table 5: Recommended project timeline

Late Submission Policy

Late submissions will receive a grade of 0. Extensions will only be granted for documented medical emergencies or other extenuating circumstances approved by the instructor.

13 Academic Integrity and GenAI Use Policy

13.1 Generative AI Use Policy

You MAY use GenAI tools for:

- Grammar and spelling correction in your report
- Coding (you must be able to explain the code)
- Documentation assistance (e.g., improving code comments)
- Generating ideas for visualization approaches

You MAY NOT use GenAI tools to:

- Write substantial portions of your report
- Conduct your analysis

13.2 Required Disclosure

You must disclose all GenAI usage in:

1. A dedicated section at the end of your report (before references)

Disclosure Format:

Generative AI Use

We used [Tool Name] for [specific purpose] in [specific location].

Example: “We used ChatGPT to improve grammar in the introduction section and to debug a data loading error in data_loading.py line 45.”

13.3 Verification

- You must retain all prompts used with GenAI tools
- The instructor may request to review prompts at any time
- You must be able to explain and defend every aspect of your work

When in doubt about how to use Generative AI

Ask the instructor.

14 Frequently Asked Questions

1. **Q: How much time should this project take?**

A: Plan for approximately 100 hours per team (50 hours per person) spread over 50 days.

2. **Q: How should we split the work?**

A: This is up to your team. However, both members should contribute equally.

3. **Q: What programming language must we use?**

A: Python.

4. **Q: Can we use open-source models?**

A: Yes, you may use open-source models as long as you fine-tune them on the project data and clearly document this in your report.

5. **Q: Can we use other OPF codes and tools?**

A: All analysis must be reproducible via Python code. However, you may use tools like Panda Power or other variations of AC OPF for exploration, as long as your final deliverables are generated from Python code in your repository.

6. **Q: What if we can't beat the benchmark?**

A: Beating the benchmark is not the only criterion. If you implement a sophisticated approach with thorough analysis showing why it didn't beat the benchmark, you can still receive the grade. However, you must demonstrate technical competence and rigorous evaluation.

7. **Q: Can we try multiple approaches?**

A: Yes.

8. **Q: How do we handle constraint violations?**

A: Your approach must include a strategy for constraint handling. Options include:

- Post-processing projection
- Penalty methods in training
- Safe RL methods

Document your strategy and analyze constraint violation rates.

9. **Q: What computational resources are available?**

A: The project is designed to run on standard laptops without the need for GPU.

10. **Q: Can we use data from outside the provided dataset?**

A: No. You must use only the provided dataset for fair comparison with the benchmark.

11. **Q: What if our forecasting is poor but OPF is good?**

A: Your final evaluation is based on overall system performance. If poor forecasting leads to poor dispatch decisions, this will be reflected in results. You should analyze this trade-off.

12. **Q: Can we modify the benchmark?**

A: No. The benchmark is fixed for fair comparison. However, you can implement additional baselines for comparison in your analysis.

13. **Q: Can we discuss our work with other teams?**

A: Actually encouraged; You may discuss general approaches and technical issues, but your analysis, visualizations, and report must be your team's original work.

15 Data Confidentiality and Research Collaboration

Important: Data Confidentiality

The PV dataset provided for this assignment is part of ongoing research at the Distribution Grids Research & Innovation (DGRI) Lab. This data is **confidential and must not be shared, published, or released** outside of this course in any form. Students are strictly prohibited from:

- Sharing the PV dataset with anyone not enrolled in this course
- Posting the data or any derived results on public repositories, websites, or social media
- Using the data for purposes other than completing this assignment
- Retaining the data after the course concludes

Opportunity for Research Contribution

Considering that the provide code and data are processed by the DGRi Lab, if you create particularly innovative or insightful methods, you may seek instructor's approval for publications. Note that any intention to publish the findings:

- Require all parties explicit consent
- Follow appropriate co-authorship or acknowledgement protocols

This is not a requirement for the assignment grade, but represents an opportunity for students to contribute to peer-reviewed research and gain research experience.

Good luck with your project!

This is a challenging but rewarding opportunity to work on cutting-edge research at the intersection of machine learning and power systems.

This project addresses course learning outcomes 1, 2, 3, 4, 5, and 6 of the course outline.